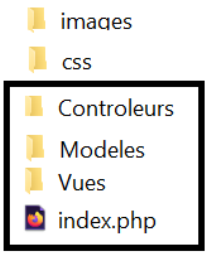
Liens vers le référentiel		
	Bloc 1	Support et mise à disposition des services informatiques
	Compétence 1.3	Développer la présence en ligne de l'organisation
	Sous-compétence	Participer à l'évolution d'un site Web exploitant les données de l'organisation
	Notions	Langage d'interrogation de données - SQL partie LID :
	Bloc2	Conception et développement d'applications
	Compétence 2.1	Conception et développement d'une solution applicative
	Sous-compétence	Intégration continue des versions d'une solution applicative
	Bloc2	Conception et développement d'une solution applicative
	Compétence 2.1	Participer à la conception de l'architecture d'une solution applicative
	Sous-compétence	Architecture MVC – Modèle Vue Controleur

L'architecture **MVC (Modèle – Vue – Contrôleur)** est un **patron de conception** (design pattern).
Un patron de conception est un modèle ayant fait ses preuves par le passé et qui est considéré comme une bonne pratique pour un objectif spécifique.
Les avantages d'un patron de conception sont :

- fournir une structure standard qu'il suffit d'adapter à son besoin
- accélérer le processus de développement
- augmenter la qualité (code optimisé et fiabilité)
- faciliter la maintenance grâce à une structure connue

1. Principe

Dans l'architecture MVC, l'organisation des sites web ressemble à ceci :

	Le point d'entrée est toujours le fichier ***index.php***, il joue le rôle d'aiguilleur dans le site. Le dossier ***Vues*** contient toutes les pages fournissant le contenu à afficher (interface graphique) Le dossier ***Controleurs*** contient les fichiers qui vont contenir les règles métier du site (spécifiques aux données et procédures de l'entreprise) Le dossier ***Modeles*** contient les fichiers interagissant avec les données stockées (Base de données, fichiers XML, fichiers Json, …)

2. Les 3 composants

2.1. Les vues

1.1.1. Vue externe

L'aperçu global du site visible par l'utilisateur s'appelle la vue externe. Techniquement, elle est obtenue à partir de plusieurs vues assemblées, par exemple : l'en-tête, le contenu principal et le bas de page.

1.1.2. Sous-vues

Il s'agit des différentes pages permettant de construire la vue externe visible par l'utilisateur. En général, une partie de la vue externe est identique sur l'ensemble de la navigation et seule une zone change en fonction de la demande de l'utilisateur. Cette zone variable est une vue adaptée à l'action spécifiée en paramètre et réalisée par le sous-contrôleur spécifique.

3. Les contrôleurs

3.1. Contrôleur principal

La page index.php joue le **rôle d'aiguilleur.** Ceci signifie qu'elle va orchestrer le travail et/ou l'affichage des composants du site pour répondre à la demande de l'utilisateur. On l'appelle le contrôleur principal.
Dès que l'utilisateur clique sur une fonctionnalité, l'url du site se complète afin de donner les informations nécessaires à la page index.php.
L'url respecte systématiquement la structure suivante :

http://adresseDuSite/index.php**?controleur=gestionAgence&action=consulterLesAgences**

L'url comporte 2 paramètres passés par la méthode **GET** :

- **controleur** : indique quel élément du site est concerné (en général, un élément correspond à une table de la base de données métier)
- **action** : indique l'opération souhaitée par l'utilisateur sur l'élément passé en premier paramètre (en général : consultation, ajout, suppression, modification)

Le contrôleur principal dirige la demande au sous-contrôleur spécifique selon la valeur du paramètre « controleur » .

3.2. Sous-contrôleurs

Chaque sous-contrôleur, appelé par le contrôleur principal, va orchestrer le travail nécessaire à la réalisation de l'action passée en paramètre (valeur du paramètre action). Ceci peut nécessiter :

- la récupération de données via les modèles
- l'affichage de résultats à l'utilisateur via les vues

4. Les modèles

Il s'agit des fichiers permettant d'accéder aux données nécessaires à l'application web. Ces fichiers contiennent la connexion mais également toutes les méthodes utiles aux sous-contrôleurs pour réaliser les actions **- sur les données -** demandées par l'utilisateur.

L'avantage de ce composant est de rendre l'application moins dépendante du support de stockage des données. Ainsi, si les données sont stockées dans une base de données relationnelles, puis que plus tard, elles sont basculées dans des fichiers xml, seule la partie Modèles sera à adapter. Le reste de l'application restera inchangé.

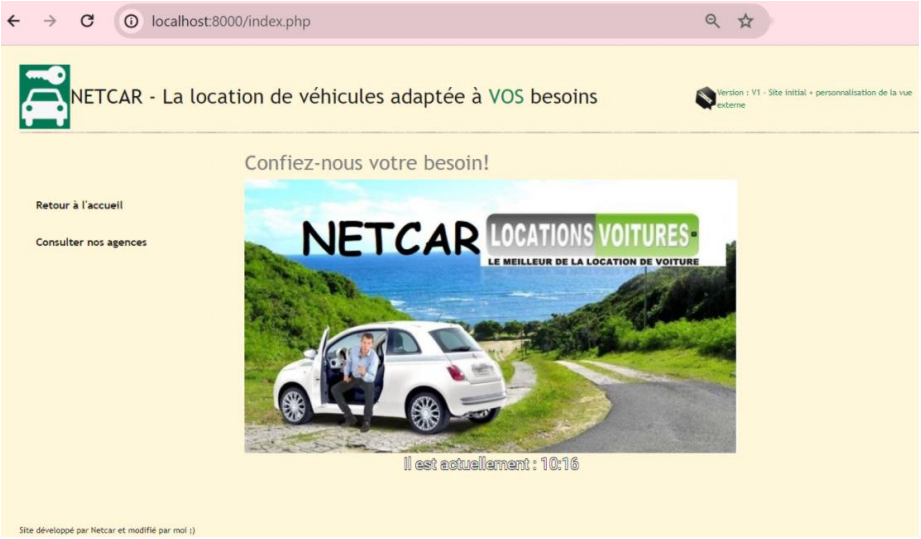
5. Observation détaillée du mécanisme

5.1. Préparer le projet et tester son fonctionnement

1. Lancer Netbeans et ouvrir le projet **T:\AP2\AP2.2ProjetPHPNetcarMVC** (cf séance précédente)

2. Lancer WampServer (il est conseillé de se replacer sur le même ordinateur afin de ne pas à avoir à reparamétrer votre projet et à réimplanter la base de données).

3. Vérifier le bon fonctionnement du site.



6. Analyser la structure du site fourni

4. Ouvrir le fichier **index.php** dans Netbeans et répondre aux questions :

<table>
<tr>
<td>
a. La vue externe affichée par ce fichier est obtenue grâce à l'inclusion de plusieurs fichiers.
b. Citer le nom de ces fichiers (voir les instructions require_once et include)
c. Indiquer la zone affichée par chaque fichier
<pre>require_once("Modeles/ConnexionBD.php");

//import des instructions permettant de construire le bandeau d'en-tête
include("Vues/v_layout_entete.php") ;</pre>
</td>
<td>

a) C_gestionAgence.php
b) Modeles/Connexion.php et v_layout_entete.php
c) Index.php : affiche une entete qui affiche la marque de l'agence, un slogan et la version du fichier et un pied de page qui affiche qui a développé le site, le logo de l'agence, un texte pour afficher l'heure ; consulter nos agence qui se trouvent dans v_consulterLesAgences.php
</td>
</tr>
<tr>
<td>
d. A quoi sert la structure alternative 'if' présente dans ce fichier ?
<pre>if (!isset($_GET['controleur'])) {
    $controleur = 'accueil';
    $stylePresentation = "";
    $message = "";
}
else {
    $controleur = $_GET['controleur'];
}</pre>
</td>
<td>
C'est une condition qui affiche si la page du dossier
</td>
</tr>
<tr>
<td>
e. A quoi sert la structure alternative 'switch' présente dans ce fichier ?
<pre>switch($controleur)
{
        case 'accueil':
                { include("vues/v_accueil.html");
                  break;
                }

        case 'gestionAgence' :
                { include("controleurs/c_gestionAgence.php");
                  break;
                }
}</pre>
</td>
<td>
L'instruction switch permet d'inclure des fichiers
</td>
</tr>
<tr>
<td>
f. Pourquoi est-il nécessaire de mettre l'instruction break à chaque fin de bloc 'case' ?
</td>
<td>
L'instruction break permet de sortir d'une condition
</td>
</tr>
</table>

```
switch($controleur)
{

        case 'accueil':
                { include("vues/v_accueil.html");
                  break;
                }

        case 'gestionAgence' :
                { include("controleurs/c_gestionAgence.php");
                  break;
                }
}
```

5. Ouvrir, dans Netbeans, le fichier **vues\v_layout_entete.php** (vue)

Indiquer le rôle des instructions situées aux lignes : 5, 12, 14, 23-24, 27-28, 37-38

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  <meta charset="utf-8" />
5  <meta name="viewport" content="width=device-width, initial-scale=1" />
6
7  <title>NETCAR</title>
8  <!--
9  <link href="bootstrap/css/bootstrap.min.css" rel="stylesheet" type="text/css">
10 -->
11
12 <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.1/dist/css/bootstrap.min.css" rel="stylesheet" integrity="sha384-+0n0xVW2eSR5OomGNYDnhzAbDsOXxcvSN1TPprVMTNDbiYZCxYbOOl7+AMvyTG2x" crossorigin="anonymous">
13
14 <link href="/css/netcar.css" rel="stylesheet" type="text/css">
15
16
17 </head>
18 <body>
19
20 <div id="blocPage">     <!-- à ajouter  : document entier-->
21         <header>                                <!-- à ajouter  : en-tête-->
22             <div id="entete" class="h1">
23                 <div> <img id="icone" src="./images/netCar_icone.jpg" alt="NetCar_icone" /> </div>
24                 <div> NETCAR - La location de véhicules adaptée à <span class="text-success"> VOS </span> besoins </div>
25             </div>
26             <div id="version" class="text-success">
27                 <div> <img src="images/netCar_epingle.png" alt="NetCar_epingle" /> </div>
28                 <div> Version : V1 - Site initial + personnalisation de la vue externe </div>
29             </div>
30
31         </header>
32
33         <section>
34         <div id="leMenu">
35             <nav class="">
36             <ul>
37                 <li><a href="index.php"> Retour à l'accueil</a></li>
38                 <li><a href="index.php?controleur=gestionAgence&action=consulterLesAgences">Consulter nos agences</a></li>
39             </ul>
40             </nav>
41         </div>
42         <!-- contenu de droite définie dans les vues id="contenu"-->
```

Ligne 5 :
Ligne 12 :
Ligne 14 :
Ligne 23-24 :
Ligne 27-28 :
Ligne 37-38 :

6. Ouvrir le fichier **controleurs\c_gestionAgence.php**

```
<?php
include 'Modeles/AgenceDAO.php';
$objAgence= new AgenceDAO();
if(!isset($_GET['action']))
    $action = 'consulterLesAgences';
else
    $action = $_GET['action'];

switch($action)
{
        case 'consulterLesAgences':
        {
            $lesAgences = $objAgence->getLesAgences();

            include("vues/v_consulterLesAgences.php");

            $message = "Netcar comporte ". count($lesAgences). " agences pour être au plus proche de vous.";
            $stylePresentation = "text-success";
            break;
        }

}
```

a. A quoi sert la structure alternative 'if' présente dans ce fichier ?	
b. A quoi sert la structure alternative 'switch' présente dans ce fichier ?	
c. Décrire les instructions appelées dans le case 'consulterLesAgences'	
d. Retrouver où les variables $message et $stylePresentation sont utilisées par la suite. e. A l'aide d'internet si besoin, trouver où est défini la classe CSS « text-success ». Retrouver comment cette bibio-thèque est intégrée dans le projet Netcar.	

Bootstrap est un framework CSS. Un framework correspond à un ensemble de librairies regroupées dans un but précis et possédant des règles internes que doivent suivre les utilisateurs.

En d'autres termes, et pour le dire très simplement, Bootstrap est un ensemble de fichiers CSS et JavaScript fonctionnant ensemble et qu'on va pouvoir utiliser pour créer des design complexes de manière relativement simple.

Pour utiliser Bootstrap, nous allons donc simplement devoir utiliser ces fichiers. On va pouvoir faire cela de deux façons. On peut :
- soit **télécharger les fichiers Bootstrap (CSS et JavaScript)** sur le site https://getbootstrap.com/ puis les lier à nos fichiers HTML comme n'importe quel autre fichier CSS et JavaScript ;
- soit **utiliser un CDN** (**Content Delivery Network** ou réseau de distribution de contenu) et inclure le lien vers les fichiers dans nos fichiers HTML.

C'est cette seconde solution qui a été choisie. Cependant, les fichiers ont été aussi importés pour pouvoir tester les deux méthodes de fonctionnement.

7. Ouvrir, dans Netbeans, le fichier **Modeles\ConnexionBD.php**

```
<?php
class ConnexionBD
{   private $connexion;

    // constructeur de la classe BdPdo : en PHP le constructeur se nomme toujours __construct
    public function __construct()
    {
        try{
            $this->connexion=new PDO('mysql:host=localhost;dbname=bdNetCar','root','');
            $this->connexion->query("SET CHARACTER SET utf8");
            //echo "<br/>connexion réussie.<br/>";
        }
        catch (PDOException $erreur){
            echo "Erreur de connexion à la base de données ".$erreur->getMessage();
        }
    }

    protected function getConnexion() {
        return $this->connexion;
    }
}
```

a. Combien y a-t-il d'attributs définis dans la classe (partie privée) ? b. Comment s'appelle la méthode qui permet de construire un objet (instance) de la classe ? Est-ce que la convention de nommage de cette méthode est identique à celle vue en langage java ? c. Repérer l'erreur présente dans le commentaire ligne 5. Le corriger. Enregistrer et remarquer le marqueur bleu affiché par Netbeans. Quel est son but ? Si besoin, cliquez sur le marqueur.	

d. Rappeler le rôle de l'instruction try/catch. e. Que contient la variable $connexion ? Quelle est la classe évoquée à la ligne 9 ? Combien de paramètres sont passés lors de l'instanciation de le la variable $connexion ? f. Concernant la fonction getConnexion() : - quelle est sa visibilité ? Qu'est-ce que cela signifie ? - quel est son rôle ?	

8. Ouvrir, dans Netbeans, le fichier **Modeles\AgenceDAO.php**

```
1  <?php
2  class AgenceDAO extends ConnexionBD
3  {
4      //cette classe hérite de la propriété $connexion
5      // la propriété $connexion contient un objet PDO permettant de se connecter et d'interagir
6      // avec une base de données
7
8      // cette classe a accès à l'accesseur getConnexion() qui retourne la propriété $connexion
9      // grâce à cet objet $connexion de la classe PDO, la fonction query() sera accessible
10     // la fontion query() permet de demander l'exécution d'une requête au SGBD
11
12
13     public function getLesAgences()
14     {
15         $req = "select * from agence order by nom asc;";
16
17         // $this: représente l'objet Agence dans lequel on est
18         // getConnexion() est la fonction qui retourne la propriété $connexion
19
20         $resultatDeLaRequete = $this->getConnexion()->query($req);
21
22         $tableauDesLignesResultat = $resultatDeLaRequete->fetchAll();
23         return $tableauDesLignesResultat;
24     }
25 }
26
   ?>
```

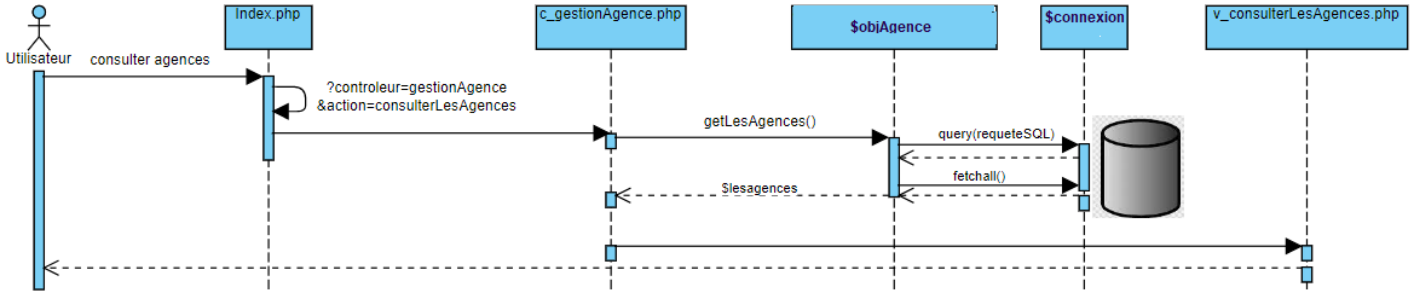
a. Repérer et comprendre l'utilité des informations inscrites à la ligne 2 (après le nom de la classe) b. Quel est le rôle de la méthode getLesAgences ? c. Décrire les instructions qui la composent.	

9. Ouvrir le fichier **vues\v_consulterLesAgences.php**

```
1  <div id="contenu">
2
3  <fieldset>
4      <legend>Nos agences </legend>
5
6          <table class="table">
7              <thead>
8                <tr>
9                  <th>Nom</th>
10                 <th>Ville</th>
11                 <th></th>
12               </tr>
13             </thead>
14             <tbody>
15                 <?php
16                 foreach($lesAgences as $lAgence) {
17                 ?>
18                     <tr>
19                         <td>
20                             <?php echo "<a href='index.php?controleur=gestionAgence&action=consulterLesHoraires&agence=";
21                                 echo $lAgence['CODEAGENCE']."'>". $lAgence['NOM']."</a>";?></td>
22
23                         <td><?php echo $lAgence['VILLE'];?></td>
24                         <td>
25                             <?php echo "<a href='index.php?controleur=gestionLocation&action=faireUneLocation&agence=".$lAgence['CODEAGENCE']."'>";?>
26                                 <button class="btn btn-success" type="button">Louer un véhicule</button>
27                           <?php echo"</a>"; ?>
28                         </td>
29                     </tr>
30                 <?php
31                 }
32                 ?>
33             </tbody>
34         </table>
35 </fieldset>
36 </div>
```

a. Quel est le rôle de la balise <table> ?	
b. Quel est le rôle du foreach ?	
c. Expliquer l'instruction de la ligne 20-21	
d. Quelle est la nature de l'objet $lAgence ?	
e. Peut-on indiquer n'importe quelle valeur pour l'étiquette située entre les crochets ?	

6. Représentation du mécanisme avec un diagramme de séquence (UML)



7. Test des fonctionnalités et observation de la modification de l'url

a) Noter les url après chaque action ci-dessous :

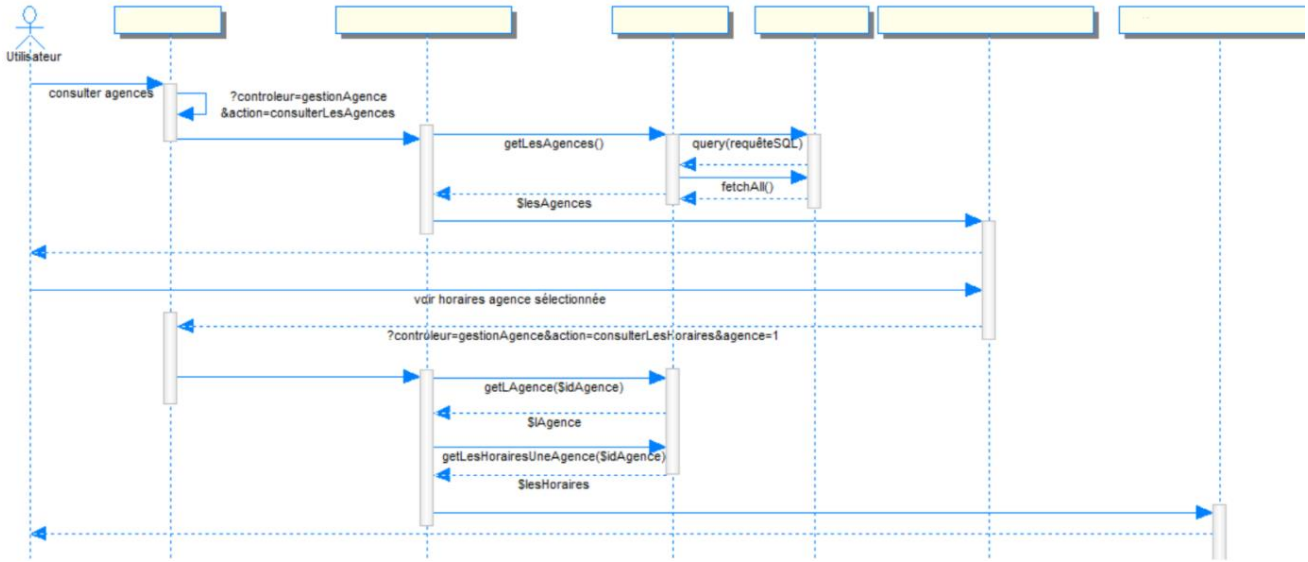
Clic sur « Accueil »	http://localhost/projetphpnetcarmvc**/index.php**
Clic sur « Consulter les Agences »	
Puis Clic sur « La nantaise »	
Retour sur « Les Agences » puis Clic sur « Louer un véhicule » <u>sur la première ligne</u>	

b) Pour chaque url ci-dessus, retrouver le « circuit » réalisé dans les composants du site pour déterminer le bloc d'instructions à réaliser : s'appuyer sur le code des différents fichiers appelés.

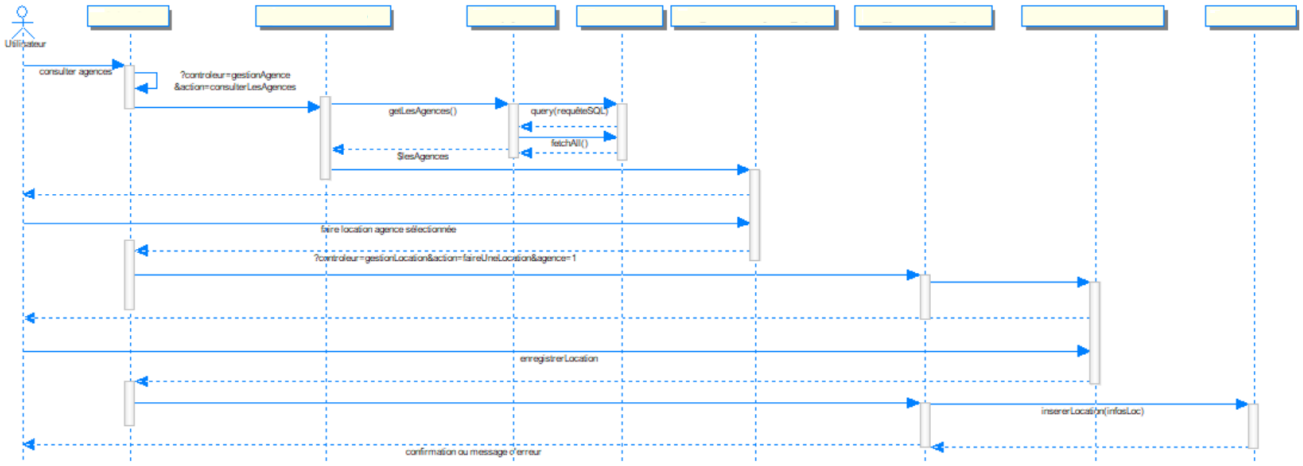
http://localhost/projetphpnetcarmvc**/index.php**	
...index.php?controleur=gestionAgence&**action=consulterLesAgences**	
....index.php?controleur=gestionAgence&**action=consulterLesHoraires&agence=1**	
...index.php?controleur=gestionLocation&**action=faireUneLocation&agence=1**	

c) Compléter le diagramme de séquences complet des différentes fonctionnalités

<u>Consulter les horaires d'une agence :</u>



Faire une location dans une agence :



d) En fonction des fichiers déjà présents et de leur contenu, en déduire les éléments à rajouter pour que les fonctionnalités ci-dessus fonctionnent.

=> contrôleur à créer : ……

⇒ actions à définir dans nouveau contrôleur : ……

⇒ actions à rajouter dans contrôleur existant: ……

⇒ modèle DAO à rajouter : ……

⇒ méthode à définir dans ce modèle DAO : ……..

⇒ méthodes à rajouter dans le modèle DAO existant :